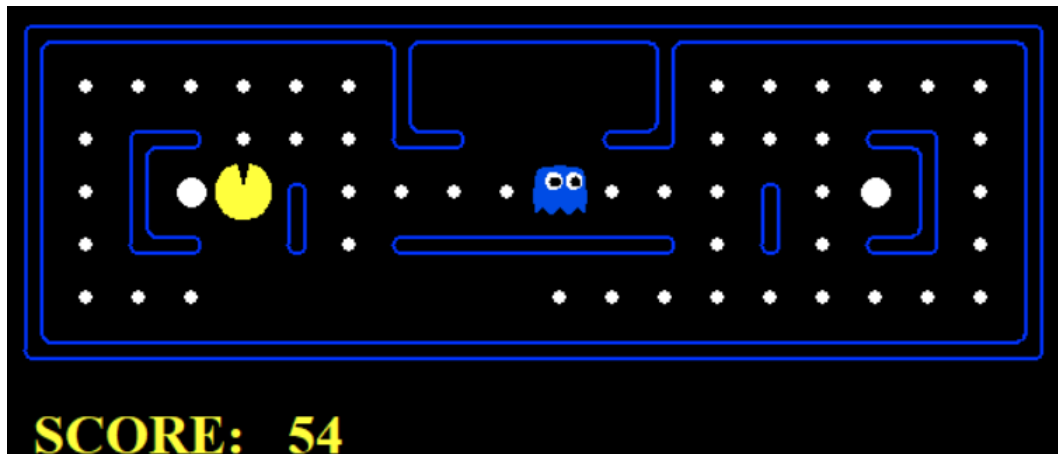


FIT3080

Assignment 1: Problem Solving as Search [64 marks]



Introduction

In the arcade game of *Pac-Man*, an eponymously-named agent accumulates points by collecting food dots while avoiding ghosts along the way. In the original version of the game, Pac-Man is controlled by a human player and ghosts are controlled by the environment. In this assignment, we will create a computer controller to play Pac-Man as efficiently as possible. We will consider different variations of the classic game to study different topics involving state-space search.

There are three parts to this assignment:

- **Part 1** is about single-agent search. You will tackle a number of different maze navigation challenges that ask you to collect dots as efficiently as possible.
- **Part 2** is about adversarial search. You need to program a computer controller that plays a real game of Pac-Man, with a single ghost.
- **Part 3** involves an in-class Question & Answer (Q&A) component. This component assesses your understanding of your own solution, including your design decisions, implementation approach, and interpretation of the results.

This assignment is worth **20% of your marks** for this unit. Please check the marking rubric provided on Moodle for more details. You will be evaluated based on:

1. The performance of your controller on the various game challenges (**75% of the assignment marks**).
2. The quality of your Q&A answers, which describe and justify your implemented techniques (**25% of the assignment marks**).

The due date for this assignment is **Monday, September 7, 2026 at 11:55pm** (Melbourne local time). The Q&A component will take place during the following applied class between September 9-11, 2026.

SUBMISSION INSTRUCTIONS for this assignment are described on the last page of this document. Please follow these instructions carefully.

Pac-Man Overview

We will use a simulated version of the Pac-Man game, developed at the University of California, Berkeley, which is written in Python. Instructions for how to download and install this software can be found in our **Installation Guide**. Descriptions of the software and tips for how to tackle the assignment can be found in our **Getting Started Guide**.

In broad strokes:

1. The state of the game is tracked and advanced by a Pac-Man **game simulator**.
2. The simulator divides a game into a sequence of discrete **iterations** or timesteps.
3. In each iteration, a **controller** object must specify valid actions for the pacman agent in the game. The possible actions are: **Up, Down, Left, Right** and **Stop**.
4. Every action has a **cost of 1**.
5. Given a valid action, the game simulator **updates** the position of the pacman agent and, if necessary, updates the score (invalid actions have no effect).
6. The game simulator continues iterating until one of the following **termination conditions** are reached:
 - The game board has been cleared of all food dots (i.e., the problem is solved).
 - Pac-Man is caught by a ghost (i.e., the game is over, since Pac-Man only has a single “life”).
 - Time runs out.

You will implement several Pac-Man controllers, each solving a different kind of problem. Solution quality is measured in terms of **path length** (shorter is better). Each time you solve a problem, the corresponding **plan** (i.e., the sequence of actions computed by you) is printed to the terminal (stdout).

Support

If you have questions about any part of this assignment specification, the starter code or its supporting documentation, please reach out to the teaching team. We suggest Ed as a first stop, followed by email and then consultations. We will do our best to respond to your questions as effectively as possible.

Submission

PAY CAREFUL ATTENTION TO THE INSTRUCTIONS IN THIS SPECIFICATION

- Remember to submit all required files.
- Do not modify any code that we do not ask you to.
- Each submission must be your own work (you may seek the assistance of an AI, but you should declare how you used the AI).

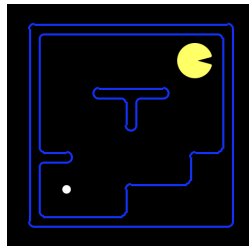
Failure to comply with these instructions may result in your attempt receiving an overall score of ZERO.

Part 1: Single Agent Search [24 marks]

In this part of the assignment, we will develop an AI controller to play *Pac-Man: Collect the Dots*, a single-agent variant of Pac-Man. In this game, you must navigate a series of maze layouts, collecting as many dots as possible to maximise your score. There are no ghosts.

Question 1(a) [5 marks]

In this question, Pac-Man must find a path through the maze to collect a single dot:



1. Define the problem (i.e., game state, etc.) in the `q1a_problem` class.
2. **Implement the A* algorithm** using the Manhattan distance heuristic in the `q1a_solver` class. Please only use the default Python modules, `numpy` and `scipy`. We will not be providing any others.

To solve this problem, your agent will have access to: the size of the maze, the location of every wall in the maze, the starting location of Pac-Man, and the location of the food dot. For more detail, please read Section 2 of the **Getting Started Guide** about the `GameState` class.

The command below shows how you can test your algorithm. Modify the `-l` parameter to try different maze layouts located in the `layouts` directory with the prefix `q1a_`.

```
> python pacman.py -l layouts/q1a_tinyMaze.lay -p SearchAgent -a
    fn=q1a_solver,prob=q1a_problem --timeout=1
```

Marking

There are 5 marks available for the computational part of this question. Your `q1a_solver` will be evaluated on 5 instances with a **timeout of 1 second**. **Timing out on any instance will result in 0 marks for that instance.**

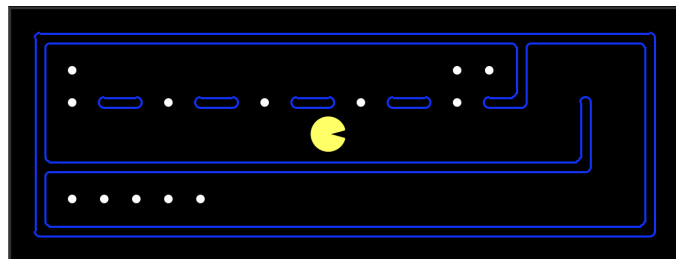
When your solver returns a plan for each instance, the game simulator will execute the actions one-by-one. The game finishes when there are no more actions to execute. If your Pac-Man agent fails to collect the dot, you will receive a score of 0 for this instance. If your Pac-Man successfully collects the dot, you will receive a mark of

$$\frac{C^*}{C}$$

for that instance, where C is the cost of your solution and C^* is the minimum possible cost. We compute these minimum costs using a simple implementation of an A* algorithm using the Manhattan distance heuristic. The mark for each instance is in the range $[0, 1]$, **allowing a maximum of 5 marks for this question**. Full marks should be achievable.

Question 1(b) [19 marks]

This question is similar to Q1(a), but there are now multiple dots in the maze. Collect as many dots as possible to maximise your score, but consider that some dots might be so far apart that they're not worth collecting. Also note that it is possible for your final game score to be negative if the number of steps Pac-Man takes is greater than the positive score received from collecting food dots. You will need to strategically balance the amount of search against the solution quality. This means you should design your agent to balance the computational effort spent searching (e.g., planning time or number of states explored) against the quality of the resulting solution. A better-quality solution (e.g., collecting more food and achieving a higher score) is desirable, but not if it requires an excessive amount of search compared to a slightly lower-quality solution that can be found much more efficiently.



1. Define the problem (i.e., game state, etc.) in the `q1b_problem` class.
2. Implement **any of the algorithms in module LN3** to solve this problem in the `q1b_solver` class. You may modify the algorithm, but the base algorithm must be from LN3.

To solve this problem, your agent will have access to: the size of the maze, the location of every wall in the maze, the starting location of Pac-Man, and the location of all food dots. For more detail, please read Section 2 of the **Getting Started Guide** about the `GameState` class.

The command below shows how you can test your implementation. Modify the `-l` parameter to try different maze layouts located in the `layouts` directory with the prefix `q1b_`.

```
> python pacman.py -l layouts/q1b_tinySearch.lay -p SearchAgent -a
    fn=q1b_solver,prob=q1b_problem --timeout=10
```

Marking

There are 19 marks available for the computational part of this question. Your `q1b_solver` will be run on 19 evaluation instances. Each run has a **timeout of 10 seconds**. **Timing out on any of these instances will result in 0 marks for this instance**. Otherwise, we will calculate your mark based on the score shown in the game GUI, measured relative to the score achieved by an existing benchmark algorithm. This algorithm is reasonably strong, but obtaining a good score for this question should be achievable.

If you match the score achieved by this algorithm, then you will receive 1 (full) mark for that instance. If you beat the algorithm you will receive up to 10% bonus marks. Your mark for this part will be calculated using the following formula:

$$\min \left(\max \left(\frac{S}{\hat{S}}, 0 \right), 1.1 \right)$$

where $\hat{S}$ is the score of the algorithm and S is your score. This formula restricts your mark on each instance to the range $[0, 1.1]$, and hence you will not be penalised further if your score on any instance is negative.

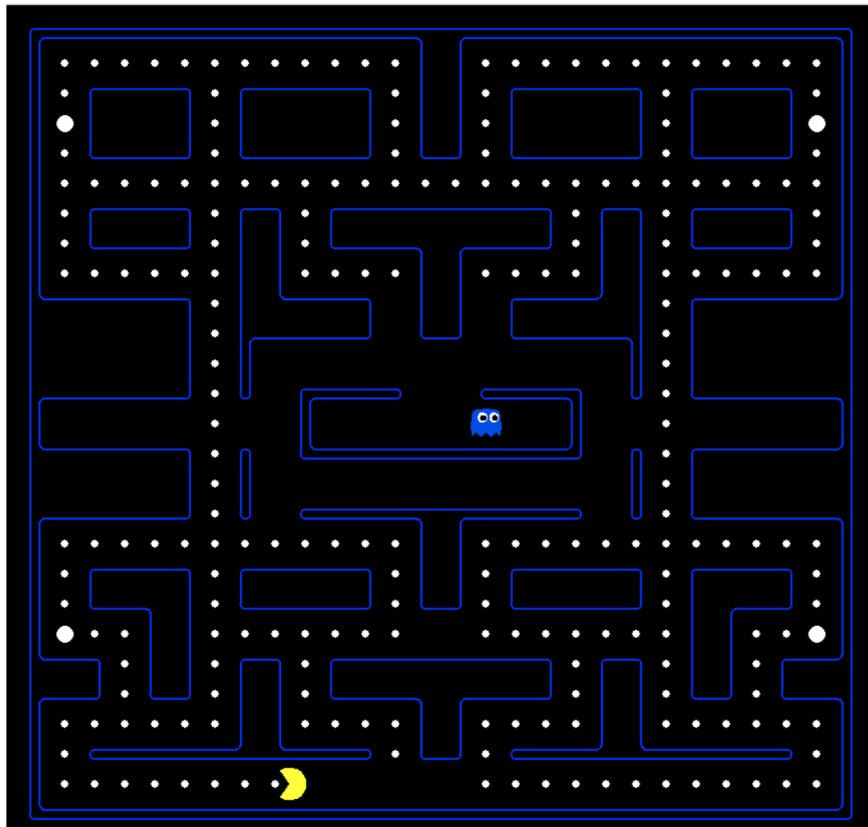
Please note that the marks in Question 1(b) can exceed the nominated total, but the total mark for the assignment cannot exceed 64 marks.

Part 2: Adversarial Search [24 marks]

In this part of the assignment, you will develop a controller that plays an actual game of Pac-Man. You must collect dots for points, but now there is a ghost to avoid.

Question 2 [24 marks]

There are multiple dots throughout the maze and a ghost. In addition to the information for Part 1, you will also have access to the number of ghosts in the maze (1 for this assignment). The ghost is controlled by the game environment.



Implement a controller for Pac-Man based on the α - β algorithm in the `getAction` function in the class `Q2_Agent`. During each iteration of the game, the `getAction` function will receive the current game state and perform an adversarial search. Pac-Man is the maximiser agent (i.e., α) and the ghost is a minimiser agent (i.e., β). Once your search is finished, you should return the most promising action for Pac-Man to execute. You are free to customise your algorithm, provided that you base it on α - β search.

You can use the command below to run your code. Modify the `-l` parameter to try different maze layouts located in the `layouts` directory with the prefix `q2_`.

```
> python pacman.py -l layouts/q2_testClassic.lay -p Q2_Agent --timeout=30
```

Marking

There are 24 marks available for the computational part of this question. Your `Q2_Agent` will be evaluated on 24 instances, each with a **time limit of 30 seconds**. This question is marked using

the same criterion as Question 1(b). Your mark for each instance is your score as a percentage of the score achieved by an existing benchmark algorithm, but capped to the range $[0, 1.1]$, which includes a 10% bonus.

Please note that the marks in Question 2 can exceed the nominated total, but the total mark for the assignment cannot exceed 64 marks. The benchmark algorithm is reasonably strong, so obtaining full marks on this question will be challenging.

Part 3: Question and Answer – In-class Reflection [25 marks]

The Q&A component contributes **16 marks** of the total assignment marks and will be completed **during your scheduled applied class**. This component is designed to assess your understanding of your own solution.

During the session, you will answer a set of written questions about your submitted assignment. The questions are intended to help your tutor understand:

- your overall approach to solving the problem;
- the rationale behind your design and implementation decisions;
- any heuristics, evaluation functions, or other techniques used (where applicable), and why they were chosen;
- how you interpret and evaluate your results; and
- possible improvements, alternative approaches, or future extensions to your solution.

The questions are **not designed to trick you**. If you have completed the assignment yourself and understand the decisions you made throughout the implementation, you should be well prepared to answer them.

Format

- The Q&A will be conducted **in your scheduled applied class**.
- Responses must be completed **by hand on paper**.
- **Computers, electronic devices, and online resources are not permitted** during the Q&A unless **previously** authorised.
- The questions will focus on your submitted assignment and your understanding of the techniques used, rather than requiring additional implementation.

Assessment Criteria

Your responses will be assessed holistically based on your ability to:

- clearly explain and justify your design and implementation decisions;
- demonstrate an understanding of the algorithms, heuristics, evaluation functions, or other techniques used (where applicable);
- interpret and critically analyse the outcomes of your implementation using concepts covered in the unit; and
- identify meaningful limitations, improvements, or alternative approaches.

Additional details are provided in Table 1.

Table 1: Marking rubric for Q&A component.

Performance	Design Decisions	Analysis of Results	Reflection and Improvement
Excellent	Provides clear, well-justified explanations for design choices and demonstrates a strong understanding of the selected approach in the context of the concepts taught in the unit.	Interprets results accurately, and provides meaningful insights.	Identifies meaningful limitations and proposes realistic, well-reasoned improvements or alternatives.
Good	Justifies most design decisions and demonstrates a sound understanding of the implemented solution with appropriate links to unit concepts.	Provides a reasonable interpretation of the results.	Identifies relevant improvements with appropriate justification.
Satisfactory	Describes the implemented approach with limited justification.	Summarises the results with limited analysis.	Identifies some possible improvements, although explanations may be brief or incomplete.
Unsatisfactory	Provides little or no explanation of design decisions.	Shows little understanding of the obtained results.	Unable to identify meaningful improvements or does not attempt the task.

Accessibility

Students requiring reasonable adjustments due to a disability, an ongoing medical condition, or other approved circumstances should contact **Disability Support Services** or apply for **Special Consideration** as early as possible.

Hints and Tips

- Read the accompanying **Getting Started Guide** – a gentle introduction to the Pac-Man environment and tips for how to approach the assignment tasks.
- Lecture slides and tutorial exercises for LN3 are good starting points for ideas. Check the recommended readings and optional resources for more ideas.
- Alternative text-books and related research papers (e.g., that you may find on Google Scholar) can provide you with further sources of inspiration.
- If you implement your stubs carefully, you may be able to re-use them to solve more complicated search problems or to solve subproblems.

Submission Instructions

1. Your code is automatically graded online. Follow the instructions in the **Installation Guide** on moodle and register at <http://fit3080s2.rep.monash.edu/> (NOTE: it is “HTTP”, not “HTTPS”). You will then receive an email invitation to join a Git repository on GitHub, which contains the code you need for Assignment 1.
2. **Edit the files described previously and push your code to GitHub. Make sure you do not use or edit any other file, as we will not look at them.**
3. In the website, click *Evaluate My Code*, and you will be added to the queue for evaluation. We will clone your code from your Git repository and run the grading system. You can do this at any time before the deadline, and there is no limit on how many times you can be graded. If there are errors, you will be notified. **We will retain the highest score for each of the three questions achieved across all submissions.**
4. **Upload your generative AI statement to Moodle before the deadline.**
5. **Submissions after the deadline incur late penalties at a rate of 5% per day. After 7 days, your score is zero.**

Plagiarism

You can be inspired by existing source code online but you need to acknowledge the source. **You cannot use the work of other students.**

INTENTIONAL AND UNINTENTIONAL PLAGIARISM IS NOT ACCEPTABLE, AND WILL HAVE CONSEQUENCES IF YOU ARE DEEMED TO HAVE CHEATED.

Additional resources

Generative AI can be used for this task, but you must include an **AI statement** (even if no AI was used or you found information on the Internet) in accordance with Monash Guidelines. Specifically,

- Explain which AI tools you have used and for what purposes.
- If you used generative AI tools, e.g., ChatGPT, you must acknowledge the tools and their use, note the number of iterations undertaken with each AI tool, and supply the prompts you employed.
- If you have found and used tools or resources on your own, e.g., source code or algorithms, explain why these tools were selected, and provide a URL link to them. You should also discuss the approach used in the tool.
- Describe what output from the tool/service has been included in your assignment, and where.
- Summarise how you have altered, adopted or extended the external tools you employed.

Note that you will need to be knowledgeable about the approach to do well in the Q&A component of the assignment.